MERCY LEDGER

Technical Architecture and Implementation Report

A plain-language guide for developers, reviewers, judges, and non-technical stakeholders

Prepared from the project implementation pack | 23 October 2026

How to read this report

This report explains what Mercy Ledger is designed to do, how its files work together, what calculations are performed, which external Python packages are used, and what is complete versus still planned.

The source project remains unchanged. This is documentation only. The current baseline is a starter implementation: attribution is implemented, while six backend modules and the user interface remain placeholders.

1. Executive summary

Mercy Ledger is a deterministic fund-reconciliation and decision-support prototype for cyber-fraud complaint workflows. It is intended for investigating officers, bank nodal officers, and authorized case reviewers.

The application reads structured account transactions, complaint records, dates, account information, and optional invoices. It calculates disputed-fund movement under documented assumptions and presents the result with warnings, deadlines, potential impact, and draft documents.

It does not determine guilt, authorize a freeze or seizure, replace NCRP/CFCFRMS, prove ownership of money, or provide live bank integration. Every legal or operational action remains a human responsibility.

2. The problem in simple terms

A bank account can contain legitimate opening funds, several disputed credits, and later debits. A debit may consume all or part of one victim’s funds, or it may consume a mixed pool belonging to several victims.

If this is calculated manually, reviewers can accidentally treat the whole balance as suspicious, count a transferred amount twice, or confuse a matching transaction with proof of criminal intent.

Mercy Ledger makes the calculation reproducible. It replays transactions in order, maintains explicit balances, records assumptions, and produces machine-checkable outputs.

3. System architecture

The system has four layers.

Layer 1 - Input and presets: presets.py contains supported demonstration cases. A future production input layer could validate bank statement files, but CSV upload is not implemented in the baseline.

Layer 2 - Pure analysis modules: attribution, simulator, triage, SLA, impact, grievance, and orders. These modules should perform deterministic calculations and return ordinary

Python dictionaries and lists.

Layer 3 - Pipeline: pipeline.py calls the modules in sequence, isolates stage failures, and returns an errors array so incomplete results are visible.

Layer 4 - HTTP and UI: app.py exposes FastAPI endpoints and static/index.html is intended to display the results. The current HTML is only a placeholder.

Main flow: case -> attribution -> grievance -> triage -> SLA -> impact -> orders -> API response -> browser.

4. Project file map

START_HERE.md: overall project purpose, team workflow, known limitations, and release checklist.

INTEGRATION_PROTOCOL.md: ownership boundaries and safe merge sequence.

ROLE_AND_PRIORITY_MAP.md: responsibility by person and priority.

00_RULES_FOR_EVERY_AI.md: coding constraints, safety language, determinism, and money-handling rules.

01_DATA_CONTRACT.md: frozen input fields, output keys, enums, clock semantics, and assumptions.

presets.py: the SOP-derived and illustrative cases used by the demo.

golden/: expected JSON output fixtures used for acceptance checks.

check_my_work.py: the acceptance checker and whole-pipeline referee.

pipeline.py: orchestration without web-framework logic.

app.py: FastAPI server and endpoint definitions.

static/index.html: planned operator interface; currently placeholder.

5. External modules and dependencies

The project uses only two direct third-party Python packages in requirements.txt:

1. FastAPI (fastapi>=0.110,<1.0): creates the HTTP API, declares routes, and provides the application object.

2. Uvicorn (uvicorn>=0.29,<1.0): runs the FastAPI application as an ASGI web server.

app.py also imports Pydantic symbols through FastAPI’s request-model support. Pydantic is a transitive dependency of the FastAPI stack and is used for AnalyzeRequest validation.

The FastAPI stack also brings transitive dependencies such as Starlette and AnyIO. They are installed by the package manager; the project does not directly import them in the core analysis modules.

The analysis modules are designed for the Python standard library only. They use modules such as decimal, datetime, re, copy, json, os, importlib, and sys. These are built into Python and are not external packages.

No pandas, NumPy, requests, database driver, machine-learning framework, OCR library, cloud SDK, payment SDK, or AI/LLM package is used.

Dependency summary: external direct packages = FastAPI and Uvicorn; validation support = Pydantic through FastAPI; core calculations = Python standard library.

6. Recommended Python and installation

The project code targets Python 3.10 or newer. Python 3.11 is the safest shared team version because it is mature and predictable. Python 3.14 can also be used if current FastAPI and Uvicorn versions install successfully and the checker passes.

Windows setup:

```
cd C:\Users\jagan\programs\mercy_ledger_priority_implementation_pack\BASELINE\mercy_ledger

py -3.11 -m venv .venv

.\.venv\Scripts\Activate.ps1

python -m pip install --upgrade pip

python -m pip install -r requirements.txt

python --version

python check_my_work.py attribution
```

For Python 3.14, replace py -3.11 with py -3.14. Do not mix virtual environments between Python versions.

7. Input data contract

A case contains case_id, tier, title, description, demo_today, account, opening_balance, transactions, complaints, seizure_timestamp, sla_context, and invoices.

Account fields: account_id, account_type, layer, and holder_name. Supported account types are SAVINGS, CURRENT, NODAL, POOL, and ESCROW.

Transaction fields: txn_id, timestamp, type, amount, utr, and narration. The type is CREDIT or DEBIT.

Complaint fields: complaint_id, victim_label, state, disputed_utr, disputed_amount, and complaint_timestamp.

Money is text with exactly two decimal places, for example "150000.00". Dates and timestamps are ISO text.

The current schema stores the outgoing debit identifier but not the receiving account identifier. Therefore it cannot prove a downstream next-hop account.

8. Attribution engine - the completed foundation

File: attribution.py. Public function: analyze_attribution(case). This is the only major analysis module currently implemented in the baseline.

Validation: required fields, money strings, timestamps, unique IDs, complaint-to-credit matching, complaint timing, and non-negative replay balance.

Ordering: transactions are sorted by timestamp and original row position, making equal timestamps deterministic.

Balances: the engine tracks the clean balance and disputed victim pools separately while also maintaining the full ledger balance.

Debit rule: disputed funds are consumed before clean funds. The code clearly labels this as a demo assumption because the supplied SOP does not state a universal debit-ordering algorithm.

Attribution: a debit affecting one victim is direct. A debit affecting multiple victim pools is allocated pro-rata according to their remaining balances.

Output: chronological epochs, balances, victim amounts, forwarded debit references, provisional holds, ratios, totals, assumptions, and warnings.

Arithmetic: Decimal is used instead of binary floating point. Values are rounded half-up to cents only at output time.

9. Attribution concepts with an example

Suppose an account starts with clean money, receives a disputed credit, and then sends out a debit. The engine first records the credit as a victim pool. When the debit arrives, it consumes the disputed pool under the configured taint-first assumption.

If the debit is smaller than the pool, the victim retains the difference in the account. If the debit is larger, the remaining amount is taken from clean balance after the disputed pool is exhausted.

If two victim pools remain and one debit consumes both, each victim receives a share based on the relative remaining balances. This is a transparent analytical allocation, not a legal determination.

The engine also records the debit transaction ID as forwarded_downstream information. Because the receiving account is absent from the input, this is evidence of an outgoing record only.

10. Triage module - Priority 1 implementation target

File: triage.py. Public function: triage(case, attribution, grievance=None). The current file is a placeholder and must be replaced for the MVP to pass.

Expected recommendations: TARGETED_LIEN_ONLY, DIGITAL_SUSPENSION_PENDING_EDD, or SEIZURE_S106.

The scorecard is project-defined illustrative logic. It is not an official SOP score, not a validated fraud classifier, and not a legal conclusion.

Expected output fields include recommendation, rationale, score, thresholds, metrics, factors, citations, human_signoff_required, and disclaimer.

The implementation must use supplied evidence and must avoid describing an account holder as guilty or innocent.

11. SLA module - Priority 1 implementation target

File: sla.py. Public function: compute_sla(case, as_of). The current file is a placeholder.

It creates five clocks: BANK_GRIEVANCE_7D, IO_DECISION_15D, INTERIM_CUSTODY_15D, HOLD_REVIEW_90D, and SUSPENSION_REVIEW_45D.

Each clock includes its start date, deadline, days left, status, note, and SOP reference where applicable.

No system date is allowed. The calculation must use the explicit as_of date or case demo_today.

The 45-day checkpoint must be labelled as a proposed policy and not as an SOP clause.

12. Impact module - Priority 1 implementation target

File: impact.py. Public functions: case_impact(case, attribution, triage) and aggregate_impact(items). The current file is a placeholder.

It compares blanket restriction with the selected recommendation and reports innocent capital freed, victim funds preserved, and cash locked under the demo interpretation.

Digital payment-channel suspension is not treated as cash being frozen. Targeted lien or digital-only suspension therefore has zero cash locked in this metric.

Aggregate output combines case results by recommendation and includes a per-case breakdown.

13. Grievance and draft orders - Priority 1 targets

grievance.py must perform invoice-to-ledger consistency checks. A matching invoice is not proof of innocence or transaction legitimacy.

GSTIN validation is format-only because no registry lookup is supplied.

orders.py must generate exactly three HTML drafts: restoration order, indemnity bond, and EDD packet.

Every generated document must say DRAFT, require authorized human review, and state that it is not legally validated or court-ready.

All case-derived HTML text must be escaped. The order module must present already-computed amounts rather than inventing new calculations.

14. API and pipeline implementation

pipeline.py deep-copies cases, resolves the analysis date, calls modules, and collects errors. This prevents one failed stage from hiding the reason for an incomplete response.

The response contains case metadata, attribution, triage, SLA, grievance, impact, orders, and errors. A failed section may be null, but non-empty errors must be visible.

app.py defines FastAPI routes: /api/health, /api/presets, /api/presets/{preset_id}, /api/analyze-case, /api/portfolio-impact, and /.

The API accepts either a preset_id or a complete case object. Portfolio analysis combines preset cases with generated simulator cases.

15. Determinism, security, and responsible design

Determinism: same input and seed must produce the same output. Only simulator.py may use seeded random.Random.

Money safety: no float arithmetic for rupee values. Output money is serialized as strings.

Input safety: modules must not mutate caller dictionaries. Generated HTML must escape untrusted text.

Dependency safety: no network, live external service, AI call, machine learning, OCR, or registry lookup is present.

Legal safety: outputs are decision support only. They must not authorize action or state guilt, innocence, or guaranteed recovery.

16. Validation and acceptance process

Golden JSON files define expected outputs for the supported cases. The checker compares structure and values and rejects extra or missing keys.

check_my_work.py also scans for stub markers, banned dependencies, floats, input mutation,

non-determinism, and import failures.

Required commands are: python check_my_work.py attribution, simulator, triage, sla, impact, grievance, orders, and finally all.

make_sample_data.py must only run after all required modules pass. benchmark.py must only run after its guard confirms the pipeline is real.

17. Current implementation status

PASS: attribution.py. The completed attribution engine passes its golden-case checker.

PLACEHOLDER: simulator.py. It currently returns repeated preset copies rather than deterministic synthetic cases.

PLACEHOLDER: triage.py, sla.py, impact.py, grievance.py, and orders.py. They currently load golden data or return placeholder output.

PLACEHOLDER: static/index.html. It currently contains only a basic placeholder page.

Overall status: the baseline is not yet a complete end-to-end application.

Observed checker result: attribution PASS; simulator, triage, SLA, impact, grievance, orders, and pipeline FAIL because of the placeholders.

18. Priority roadmap

Priority 1 - Core MVP: implement the six placeholder backend modules, replace the frontend, pass all checkers, generate fresh sample data, and demonstrate every preset.

Priority 2 - Judge-visible depth: add conservation invariants, real stress-test counts, review flags, victim explanations, transaction replay, and scenario cards without changing the protected attribution algorithm.

Priority 3 - Optional stretch: add controlled cross-account transaction links, sensitivity comparisons for alternative debit assumptions, and report fingerprint verification.

The project should not start Priority 2 until Priority 1 passes. Any Priority 3 feature that is not independently verified should be omitted.

19. Known limitations

The opening balance is treated as clean by the demo model. Real cases require sufficient statement history.

The debit-consumption order is an explicit assumption, not a universal legal rule.

The current data does not identify receiving accounts for outgoing transfers.

SOP-derived fixtures include documented normalization or assumed times where needed for chronology.

MERCHANT_200 is invented and must not be presented as a verified real-world case.

No output is a court-ready document or legal authorization.

20. Final takeaway

Mercy Ledger is designed around explainability rather than hidden scoring: every important number should be traceable to input rows, a documented rule, or an explicit assumption.

Its implementation foundation is strong in the attribution engine, shared contract, golden fixtures, and guarded checker. The next practical step is to complete Priority 1 without

changing those protected foundations.

For deployment or demonstration, use one pinned Python version, one virtual environment, the declared FastAPI/Uvicorn dependencies, and recorded checker output.